

Discrete Mathematics

Complete Exam Notes — Topics 1 to 3

Logic • Proofs • Set Theory

0 1

Contents

1 1 Logic and Its Applications

1.1 1Propositional Logic

1.1.1 1What is a Proposition?

Definition: Proposition

A **proposition** is a statement that is either **true** (T) or **false** (F), but *not both* at the same time.

Example: Identifying Propositions

- “ $2 + 2 = 4$ ” — **Yes**, it is a proposition (True).
- “It is raining today.” — **Yes**, it is a proposition (has a truth value).
- “What time is it?” — **No**, questions are not propositions.
- “ $x + 1 = 5$ ” — **No**, the truth depends on x (this is a *predicate*).

1.1.2 1Logical Connectives

Propositions are combined using **logical connectives**:

Name	Symbol	Read as	True when
Negation	$\neg p$	“not p ”	p is False
Conjunction	$p \wedge q$	“ p and q ”	both are True
Disjunction	$p \vee q$	“ p or q ”	at least one is True
Implication	$p \rightarrow q$	“if p then q ”	not ($p=T, q=F$)
Biconditional	$p \leftrightarrow q$	“ p iff q ”	both same truth value
Exclusive OR	$p \oplus q$	“ p xor q ”	exactly one is True

1.1.3 1Truth Tables for Each Connective

p	q	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$	$p \oplus q$
T	T	T	T	T	T	F
T	F	F	T	F	F	T
F	T	F	T	T	F	T
F	F	F	F	T	T	F

Note: The Implication Trap

$p \rightarrow q$ is **FALSE only** when p is True and q is False. In all other cases — including when p is False — it is **True**. This surprises many students. Think: “If you score 90, you get an A.” If you never score 90, the promise is never broken.

1.1.4 1Converse, Inverse, and Contrapositive

Given $p \rightarrow q$:

Name	Form	Logically equivalent to?
Original	$p \rightarrow q$	—
Converse	$q \rightarrow p$	Inverse
Inverse	$\neg p \rightarrow \neg q$	Converse
Contrapositive	$\neg q \rightarrow \neg p$	Original

Key Point

The **contrapositive** is logically equivalent to the original implication. This is very useful in proofs!

1.1.5 1Tautologies, Contradictions, and Contingencies

Definition: Tautology

- **Tautology:** Always True, regardless of truth values. Example: $p \vee \neg p$.
- **Contradiction:** Always False. Example: $p \wedge \neg p$.
- **Contingency:** Sometimes True, sometimes False. Example: $p \wedge q$.

1.1.6 1Logical Equivalences (Important Laws)

Two expressions P and Q are **logically equivalent** (written $P \equiv Q$) if their truth tables are identical.

Law	AND form	OR form
Identity	$p \wedge \mathbf{T} \equiv p$	$p \vee \mathbf{F} \equiv p$
Domination	$p \wedge \mathbf{F} \equiv \mathbf{F}$	$p \vee \mathbf{T} \equiv \mathbf{T}$
Idempotent	$p \wedge p \equiv p$	$p \vee p \equiv p$
Double Negation	$\neg(\neg p) \equiv p$	—
Commutative	$p \wedge q \equiv q \wedge p$	$p \vee q \equiv q \vee p$
Associative	$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$	(similar)
Distributive	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	(similar)
De Morgan's Laws		
	$\neg(p \wedge q) \equiv \neg p \vee \neg q$	$\neg(p \vee q) \equiv \neg p \wedge \neg q$

Key Point

De Morgan's Laws are used everywhere — in proofs, programming, and circuit design. Memorise them! They say: “negate the whole $\Rightarrow$ flip $\wedge/\vee$ and negate each part.”

1.1.7 Logical Implication (Entailment)

Definition: Logical Implication

A set of premises **logically implies** conclusion C (written $P_1, P_2, \dots \models C$) if every assignment of truth values that makes all premises True also makes C True.

1.2 Predicate Logic and Quantifiers

1.2.1 Predicates

Definition: Predicate

A **predicate** is a statement that contains *variables* and becomes a proposition when those variables are assigned values. Write $P(x)$ to mean predicate P applied to variable x .

Example: Predicate

Let $P(x)$ mean “ $x > 5$ ”.

- $P(10)$ is True.
- $P(3)$ is False.
- $P(x)$ itself has no truth value until x is specified.

1.2.2 Quantifiers

Definition: Universal Quantifier $\forall$

$\forall x P(x)$ means “For **all** x , $P(x)$ is True.” It is False if even **one** value of x makes $P(x)$ False.

Definition: Existential Quantifier $\exists$

$\exists x P(x)$ means “There **exists** at least one x such that $P(x)$ is True.” It is True if **even one** value of x makes $P(x)$ True.

Example: Using Quantifiers

Let the domain be all integers.

- $\forall x (x^2 \geq 0)$ — **True** (squares are non-negative).
- $\exists x (x^2 = 4)$ — **True** (take $x = 2$ or $x = -2$).
- $\forall x (x > 0)$ — **False** ($x = -1$ is a counterexample).

1.2.3 Negating Quantifiers (De Morgan for Quantifiers)

Theorem: Negation of Quantifiers

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$

$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

Note: How to Remember

“Not everyone likes coffee” = “Someone does not like coffee.”

“No one passed” = “Everyone failed.”

1.2.4 Nested Quantifiers

When quantifiers appear one after another, **order matters!**

Example: Order of Quantifiers Matters

Domain = all real numbers.

- $\forall x \exists y (x + y = 0)$: For every x , we can find a y (namely $-x$). **True.**
- $\exists y \forall x (x + y = 0)$: There is one fixed y that works for all x . **False!**

Key Point

$\forall x \exists y$ and $\exists y \forall x$ have very different meanings. Always read left to right. Never swap quantifiers of different types without changing the meaning.

1.3 Computer Architecture Applications — Boolean Algebra

1.3.1 Connection between Logic and Circuits

In digital computers:

- **T** = 1 (high voltage) and **F** = 0 (low voltage).
- Logical connectives correspond to **logic gates**.

Gate	Symbol	Boolean expression
NOT	Bubble/triangle	$\bar{A}$ or $\neg A$
AND	D-shape	$A \cdot B$ or $A \wedge B$
OR	Curved shape	$A + B$ or $A \vee B$
NAND	AND + bubble	$\overline{A \cdot B}$
NOR	OR + bubble	$\overline{A + B}$
XOR	Curved + extra arc	$A \oplus B$

1.3.2 1Boolean Algebra Laws

Boolean algebra uses the same laws as propositional logic:

$$\overline{A + B} = \bar{A} \cdot \bar{B} \quad \overline{A \cdot B} = \bar{A} + \bar{B} \quad (\text{De Morgan's Laws})$$

1.3.3 1Functional Completeness

Definition: Functionally Complete Set

A set of connectives is **functionally complete** if every Boolean function can be expressed using only those connectives.

Theorem: Functionally Complete Sets

- $\{\neg, \wedge\}$ and $\{\neg, \vee\}$ are both functionally complete.
- $\{\text{NAND}\}$ alone is functionally complete: $\neg A = A \uparrow A$ and $A \wedge B = (A \uparrow B) \uparrow (A \uparrow B)$.
- $\{\text{NOR}\}$ alone is also functionally complete.

1.3.4 1Sum-of-Products (SOP) and Product-of-Sums (POS)

Definition: Sum-of-Products (Disjunctive Normal Form)

An expression is in **SOP** / **DNF** if it is a disjunction (OR) of conjunctions (ANDs):

$$(A \wedge \neg B) \vee (\neg A \wedge B \wedge C) \vee \dots$$

Each group is called a **minterm**.

Example: Building a Circuit from a Truth Table

Suppose output F is 1 only when $A = 1, B = 0$ and when $A = 0, B = 1$.

Minterms: $A\bar{B}$ and $\bar{A}B$.

SOP expression: $F = A\bar{B} + \bar{A}B = A \oplus B$ (XOR).

2 1

Proofs

2.1 1Definition of Proofs and Axiomatic Systems

Definition: Proof

A **proof** is a sequence of logically justified steps that starts from known facts (axioms or previously proved theorems) and arrives at a conclusion.

Definition: Axiomatic System

An **axiomatic system** consists of:

1. **Undefined terms** — primitive concepts accepted without definition.
2. **Axioms (postulates)** — statements accepted as true without proof.
3. **Defined terms** — concepts built from primitives.
4. **Theorems** — statements that are proved from axioms and definitions.

A good axiomatic system is *consistent* (no contradictions) and *complete* (every true statement can be proved) — though Gödel showed complete systems are impossible for sufficiently powerful mathematics.

Note: Commonly Used Axioms

- **Peano Axioms** — define the natural numbers.
- **ZFC Set Theory** — foundation of modern mathematics.
- **Field Axioms** — define real numbers.

2.2 1Rules of Inference

Rules of inference are **templates** for valid logical steps. If the premises are True, the conclusion must also be True.

Rule	Form	Plain English
Modus Ponens	$p, p \rightarrow q \vdash q$	“If if-then holds and p is true, so is q .”
Modus Tollens	$\neg q, p \rightarrow q \vdash \neg p$	“If q is false, so is p .”
Hypothetical Syll.	$p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$	“Chain of implications.”
Disjunctive Syll.	$p \vee q, \neg p \vdash q$	“Eliminate a false disjunct.”
Addition	$p \vdash p \vee q$	“OR anything in.”
Simplification	$p \wedge q \vdash p$	“Extract one conjunct.”
Conjunction	$p, q \vdash p \wedge q$	“Combine two truths.”
Resolution	$p \vee q, \neg p \vee r \vdash q \vee r$	Used in theorem provers.

Example: Modus Ponens in Action

Premises: “If it rains, the road is wet.” It is raining.

Conclusion: The road is wet.

Formally: Let $p = \text{“It rains”}$, $q = \text{“Road is wet”}$.

$p, p \rightarrow q \vdash q$. (Modus Ponens)

Example: Modus Tollens in Action

Premises: “If n is prime, $n \neq 1$.” We know $n = 1$.

Conclusion: n is not prime.

$\neg q, p \rightarrow q \vdash \neg p$. (Modus Tollens)

2.3 1Types of Proofs**2.3.1 1Direct Proof****Definition: Direct Proof**

To prove $p \rightarrow q$ **directly**: Assume p is True, then use definitions, axioms, and rules of inference to show q must also be True.

Example: Direct Proof

Theorem: If n is even, then n^2 is even.

Proof: Assume n is even. Then $n = 2k$ for some integer k .

$n^2 = (2k)^2 = 4k^2 = 2(2k^2)$.

Since $2k^2$ is an integer, $n^2 = 2 \cdot (\text{integer})$, so n^2 is even. $\square$

2.3.2 1Proof by Contrapositive**Definition: Proof by Contrapositive**

To prove $p \rightarrow q$: Instead, prove $\neg q \rightarrow \neg p$ (the contrapositive), which is logically equivalent.

Example: Proof by Contrapositive

Theorem: If n^2 is odd, then n is odd.

Contrapositive: If n is even, then n^2 is even. (Proved above!)

Since the contrapositive is true, the original statement is true. $\square$

2.3.3 1Proof by Contradiction

Definition: Proof by Contradiction (Reductio ad Absurdum)

To prove statement P : Assume $\neg P$ is True. Derive a **contradiction** (something that is always False). Since assuming $\neg P$ leads to a contradiction, P must be True.

Example: Classic Contradiction Proof

Theorem: $\sqrt{2}$ is irrational.

Proof: Assume $\sqrt{2}$ is rational. Then $\sqrt{2} = \frac{p}{q}$ where $p, q \in \mathbb{Z}$, $q \neq 0$, and $\gcd(p, q) = 1$ (fraction in lowest terms).

Squaring: $2 = \frac{p^2}{q^2}$, so $p^2 = 2q^2$.

This means p^2 is even $\Rightarrow p$ is even $\Rightarrow p = 2m$ for some m .

Then $(2m)^2 = 2q^2 \Rightarrow 4m^2 = 2q^2 \Rightarrow q^2 = 2m^2$.

So q^2 is even $\Rightarrow q$ is even.

But now both p and q are even, contradicting $\gcd(p, q) = 1$. **Contradiction!**

Therefore $\sqrt{2}$ is irrational. $\square$

2.3.4 1Proof by Cases (Exhaustive Proof)

Definition: Proof by Cases

Break the statement into **all possible cases** and prove each case separately. Every case must be covered — no gaps allowed.

Example: Proof by Cases

Theorem: For any integer n , $n^2 + n$ is even.

Case 1: n is even. Then $n = 2k$, so $n^2 + n = 4k^2 + 2k = 2(2k^2 + k)$. Even.

Case 2: n is odd. Then $n = 2k + 1$, so $n^2 + n = (2k + 1)^2 + (2k + 1) = 4k^2 + 4k + 1 + 2k + 1 = 4k^2 + 6k + 2 = 2(2k^2 + 3k + 1)$. Even.

In both cases, $n^2 + n$ is even. $\square$

2.3.5 1Proof by Mathematical Induction

Definition: Principle of Mathematical Induction

To prove $P(n)$ for all integers $n \geq n_0$:

1. **Base Case:** Prove $P(n_0)$ is True.
2. **Inductive Step:** Assume $P(k)$ is True for some arbitrary $k \geq n_0$ (the *inductive hypothesis*). Then prove $P(k + 1)$ is True.

Conclusion: $P(n)$ is True for all $n \geq n_0$.

Example: Induction: Sum Formula

Theorem: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ for all $n \geq 1$.

Base case ($n = 1$): LHS = 1; RHS = $\frac{1 \cdot 2}{2} = 1$. ✓

Inductive step: Assume $\sum_{i=1}^k i = \frac{k(k+1)}{2}$.

Show: $\sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}$.

LHS = $\sum_{i=1}^k i + (k+1) = \frac{k(k+1)}{2} + (k+1) = (k+1) \left(\frac{k}{2} + 1 \right) = \frac{(k+1)(k+2)}{2}$. ✓

By induction, the formula holds for all $n \geq 1$. □

2.3.6 1Strong Induction**Definition: Strong Induction**

Like regular induction, but the inductive hypothesis is stronger: assume $P(n_0), P(n_0 + 1), \dots, P(k)$ are **all** True, then prove $P(k + 1)$.

Example: Strong Induction: Fundamental Theorem of Arithmetic

Every integer $n \geq 2$ can be written as a product of prime numbers.

Base case ($n = 2$): 2 is prime. ✓

Inductive step: Assume every integer from 2 to k is a product of primes.

For $k + 1$: if it is prime, done. If not, $k + 1 = a \cdot b$ with $2 \leq a, b < k + 1$.

By the strong inductive hypothesis, both a and b are products of primes, so $k + 1$ is too. □

2.4 1Application in Program Correctness and Loop Invariants**2.4.1 1What is Program Correctness?****Definition: Partial and Total Correctness**

- **Partial Correctness:** If the program terminates, its output satisfies the postcondition (assuming precondition was satisfied).
- **Total Correctness:** Partial correctness *and* the program always terminates.

2.4.2 Hoare Triples

Definition: Hoare Triple

A **Hoare triple** is written $\{P\} S \{Q\}$ where:

- P is the **precondition** — what is true before executing S .
- S is the **program segment** (code).
- Q is the **postcondition** — what must be true after executing S .

It means: if P is true before S runs, then Q is true after S finishes.

2.4.3 Loop Invariants

Definition: Loop Invariant

A **loop invariant** is a condition (predicate) that is:

1. **True before the loop begins** (Initialisation).
2. **True after each iteration** (Maintenance) — if it was true before the iteration.
3. **Used with the loop's termination condition** to prove the postcondition (Termination).

Example: Loop Invariant for a Simple Loop

Goal: Prove that the following code computes $\text{result} = n!$.

```
result = 1;
i = 1;
while (i <= n):
    result = result * i;
    i = i + 1;
```

Loop Invariant: $\text{result} = i!$ (just before checking the loop condition, after updating).

More precisely, at the start of each iteration (when we check $i \leq n$): $\text{result} = (i - 1)!$ (since i is about to be used as the multiplier).

Let us use the invariant: **At the start of the k -th iteration, $\text{result} = (k - 1)!$.**

- *Initialisation:* Before the loop, $\text{result} = 1 = 0!$ and $i = 1$. ✓
- *Maintenance:* If $\text{result} = (i - 1)!$, after the body: $\text{result} \leftarrow (i - 1)! \cdot i = i!$, then $i \leftarrow i + 1$. So $\text{result} = ((i_{\text{new}}) - 1)!$. ✓
- *Termination:* Loop ends when $i > n$, i.e. $i = n + 1$. Then $\text{result} = (n + 1 - 1)! = n!$. ✓

Key Point

Loop invariants are the bridge between program code and mathematical proof. In an exam: state the invariant clearly, prove all three parts (Init, Maintenance, Termination).

3 1 Set Theory

3.1 1Russell's Paradox

3.1.1 1Naive Set Theory

In the early days, mathematicians defined sets very loosely: “*A set is any collection of objects satisfying some property P .*” — Cantor's naive definition.

The **Comprehension Axiom (Naive)**: For any property $P(x)$, the set $\{x \mid P(x)\}$ exists.

3.1.2 1Russell's Paradox

Definition: Russell's Paradox (1901)

Let $R = \{x \mid x \notin x\}$ — the set of all sets that do **not** contain themselves.

Ask: Is $R \in R$?

- If $R \in R$: then by definition, $R \notin R$. **Contradiction.**
- If $R \notin R$: then by definition, $R \in R$. **Contradiction.**

Either way we get a contradiction! This shows that *not every property defines a set*.

3.1.3 1Resolution

- **Type Theory** (Russell & Whitehead): Objects are arranged in a hierarchy of types; a set cannot contain itself because it would be at a lower type level than itself.
- **ZFC Set Theory** (most common today): Replaces the Comprehension Axiom with the *Separation Axiom*: sets can only be formed from *subsets* of already existing sets. There is no “set of all sets,” so the paradox never arises.

3.2 1Countability

3.2.1 1Set Cardinality and Bijections

Definition: Cardinality

The **cardinality** $|A|$ of a set A is the “size” of A — the number of elements. For infinite sets, cardinality is defined using **bijections** (one-to-one correspondences).

Definition: Same Cardinality

Two sets A and B have the **same cardinality** ($|A| = |B|$) if there exists a **bijection** $f : A \rightarrow B$ — a function that is both *injective* (one-to-one) and *surjective* (onto).

Property	Meaning
Injective (one-to-one)	$f(a_1) = f(a_2) \Rightarrow a_1 = a_2$ (no two inputs map to the same output)
Surjective (onto)	Every element of B is hit by some element of A
Bijjective	Both injective and surjective

3.2.2 1Countable Sets

Definition: Countably Infinite

A set A is **countably infinite** if there is a bijection $f : \mathbb{N} \rightarrow A$, i.e., its elements can be listed in a sequence: $a_1, a_2, a_3, \dots$

Definition: Countable

A set is **countable** if it is either *finite* or *countably infinite*.

Theorem: The Integers are Countable

$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ is countably infinite.

Proof: List them as $0, 1, -1, 2, -2, 3, -3, \dots$ — this is a bijection from $\mathbb{N}$ to $\mathbb{Z}$. $\square$

Theorem: The Rationals are Countable

$\mathbb{Q}$ is countably infinite.

Proof sketch: Arrange all fractions $\frac{p}{q}$ (with $q > 0$) in an infinite grid (rows by q , columns by p). Then **diagonalize** across the grid to produce a sequence that hits every fraction. $\square$

Theorem: Union of Countable Sets is Countable

If $A_1, A_2, A_3, \dots$ are all countable, then $\bigcup_{i=1}^{\infty} A_i$ is also countable.

3.3 1Uncountability

3.3.1 1The Real Numbers Are Uncountable

Theorem: Cantor's Diagonal Argument

$\mathbb{R}$ (and even the interval $(0, 1)$) is **uncountably infinite** — it cannot be put in bijection with $\mathbb{N}$.

Proof (by contradiction): Assume $(0, 1)$ is countable. Then list all its elements:

$$\begin{aligned} r_1 &= 0.\mathbf{d}_{11}d_{12}d_{13}d_{14}\cdots \\ r_2 &= 0.d_{21}\mathbf{d}_{22}d_{23}d_{24}\cdots \\ r_3 &= 0.d_{31}d_{32}\mathbf{d}_{33}d_{34}\cdots \\ r_4 &= 0.d_{41}d_{42}d_{43}\mathbf{d}_{44}\cdots \\ &\vdots \end{aligned}$$

Construct a new number $x = 0.x_1x_2x_3x_4\cdots$ where the digit x_n is chosen to *differ* from d_{nn} (e.g. if $d_{nn} \neq 5$, set $x_n = 5$; otherwise set $x_n = 6$).

Then x differs from r_n in the n -th decimal place for every n , so $x \neq r_n$ for all n .

But $x \in (0, 1)$, so it should be on the list. **Contradiction!**

Therefore $(0, 1)$ is uncountable, and so is $\mathbb{R}$. $\square$

Key Point

The key idea of Cantor's diagonal argument: Assume a complete list exists. Build a number that differs from every entry on the list at least in one digit. This number is not on the list — contradiction.

3.3.2 Cardinality Hierarchy

Set	Cardinality name	Symbol
Finite sets	Finite	$n \in \mathbb{N}$
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}$, all countably infinite sets	Aleph-naught	$\aleph_0$
$\mathbb{R}, (0, 1)$, all intervals	Cardinality of the continuum	$\mathfrak{c} = 2^{\aleph_0}$
Power set $\mathcal{P}(\mathbb{R})$	Strictly larger than $\mathfrak{c}$	$2^{\mathfrak{c}}$

Theorem: Cantor's Theorem

For any set A , the power set $\mathcal{P}(A)$ has strictly larger cardinality than A :

$$|A| < |\mathcal{P}(A)|$$

This means there is no largest infinity — there are infinitely many sizes of infinity!

3.4 Applications of Countability in Computability and Automata

3.4.1 Why Countability Matters in Computer Science

Theorem: The Number of Programs is Countable

Every computer program (in any language) is a finite string over a finite alphabet. The set of all finite strings over a finite alphabet is countable (enumerate them by length, then lexicographically). Therefore, the **set of all programs is countable**.

Theorem: The Number of Decision Problems is Uncountable

A **decision problem** is a function $f : \{0, 1\}^* \rightarrow \{0, 1\}$ (or equivalently, a subset of $\mathbb{N}$ or Σ^*). The set of all such functions has cardinality $2^{\aleph_0}$ — uncountable.

Theorem: Most Problems are Unsolvable

Since there are *countably many* programs but *uncountably many* decision problems, there must exist problems with **no program to solve them**. This is a cardinality argument — not a property of any specific problem.

3.4.2 The Halting Problem**Theorem: Halting Problem is Undecidable (Turing**

The Halting Problem: Given a program P and input I , does P halt on I ?

No algorithm can solve this for all (P, I) pairs.

Proof sketch (diagonal argument): Suppose a program H exists that decides halting. Build a program D :

- D takes a program X as input.
- D calls $H(X, X)$.
- If H says “ X halts on X ”, then D goes into an infinite loop.
- If H says “ X loops on X ”, then D halts.

Now ask: does D halt on input D ?

- If D halts on D : H would say so, causing D to loop. **Contradiction.**
- If D loops on D : H would say so, causing D to halt. **Contradiction.**

Therefore H cannot exist. The Halting Problem is undecidable. $\square$

3.4.3 Countability in Automata

Automaton / Language class	Count
Deterministic Finite Automata (DFA)	Countably many
Regular languages	Countably many (each described by a finite regex)
Context-Free Grammars / CFLs	Countably many
Turing Machines	Countably many
All languages over $\{0, 1\}^*$	Uncountably many

Key Point

Since the set of all Turing Machines is countable but the set of all languages is uncountable, there must be languages that no Turing Machine can recognise. This is the theoretical foundation of undecidability and incomputability.

3 1

Quick

Revision Summary (Topics 1–3)

Topic 1: Logic

- A proposition is a statement with a definite truth value.
- Key connectives: $\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \oplus$.
- $p \rightarrow q$ is False **only** when $p = T, q = F$.
- Contrapositive $\equiv$ original; converse $\equiv$ inverse (but not the original).
- De Morgan: $\neg(p \wedge q) \equiv \neg p \vee \neg q$; $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
- Quantifiers: $\forall$ (all must work), $\exists$ (at least one works).
- Negate $\forall$ by using $\exists$ (and vice versa), then negate the predicate.
- NAND and NOR are functionally complete by themselves.

Topic 2: Proofs

- Proof types: Direct, Contrapositive, Contradiction, Cases, Induction.
- Induction: Base case + Inductive step (assume $P(k)$, prove $P(k+1)$).
- Strong induction: Assume all of $P(n_0), \dots, P(k)$ to prove $P(k+1)$.
- Loop invariant: True initially + maintained each iteration + gives postcondition at termination.
- Hoare triple $\{P\} S \{Q\}$: if P holds before S , then Q holds after.

Topic 3: Set Theory

- Russell's Paradox: $R = \{x \mid x \notin x\}$ leads to contradiction. Solved by ZFC.
- Two sets have same cardinality iff a bijection exists between them.
- $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ are all countably infinite ($\aleph_0$).
- $\mathbb{R}$ is uncountable — proved by Cantor's diagonal argument.
- Programs are countable; decision problems are uncountable $\Rightarrow$ undecidable problems exist.
- The Halting Problem is undecidable (diagonal argument).
- Turing Machines are countable; languages are uncountable $\Rightarrow$ unrecognisable languages exist.

Discrete Mathematics

Complete Exam Notes — Topics 4 to 6

Abstract Algebra • Advanced Counting • Graph Theory

0 Contents

1 Abstract Algebra	3
1.1 Axioms of Group Theory	3
1.1.1 What is an Algebraic Structure?	3
1.2 Addition Modulo Groups	4
1.3 Multiplication Modulo Groups	4
1.4 Other Important Groups	5
1.5 Order of an Element and Order of a Group	5
1.6 Cyclic Groups	6
1.7 Lagrange's Theorem and Corollaries	7
1.8 Axioms of Ring Theory	8
1.9 Polynomial Rings	9
1.10 Axioms of Fields	9
1.11 Primitive Elements	10
1.12 Applications in Cryptography	10
1.12.1 The Discrete Logarithm Problem	10
1.12.2 Diffie-Hellman Key Exchange	11
2 Advanced Counting	12
2.1 Recurrence Relations of Recursion Algorithms	12
2.1.1 What is a Recurrence Relation?	12
2.1.2 Solving Linear Recurrences	12
2.2 Recurrence Relations of Divide and Conquer Algorithms	13
2.3 Master's Theorem and Growth of Functions	13
2.4 Generating Functions	14
2.5 Applications of Generating Functions in Counting	15
2.6 Applications of Generating Functions in Solving Recurrence Relations	16
3 Graph Theory	17
3.1 Graph Terminologies	17
3.2 Types of Graphs	18
3.2.1 Subgraphs	18
3.3 Some Common Graphs	18
3.3.1 Complete Graph K_n	18
3.3.2 Cycle Graph C_n	19

3.3.3 Wheel Graph W_n	19
3.3.4 Hypercube Graph Q_n	19
3.3.5 Complete Bipartite Graph $K_{m,n}$	19
3.4 Bipartite Graphs	20
3.5 Euler Paths and Hamiltonian Paths	20
3.5.1 Euler Paths and Circuits	20
3.5.2 Fleury's Algorithm (Finding an Euler Circuit)	21
3.5.3 Hamiltonian Paths and Circuits	21
3.5.4 Euler vs Hamiltonian: Key Comparison	22
Quick Revision Summary	23

1 Abstract Algebra

1.1 Axioms of Group Theory

1.1.1 What is an Algebraic Structure?

An **algebraic structure** is a set together with one or more operations that follow specific rules (axioms). We build up from the simplest structure (group) to richer ones (ring, field).

Definition: Binary Operation

A **binary operation** $*$ on a set G is a function $*$: $G \times G \rightarrow G$. It takes two elements of G and returns one element of G (this property is called **closure**).

Definition: Group

A **group** is a set G together with a binary operation $*$ satisfying four axioms:

1. **Closure:** For all $a, b \in G$, $a * b \in G$.
2. **Associativity:** For all $a, b, c \in G$, $(a * b) * c = a * (b * c)$.
3. **Identity:** There exists an element $e \in G$ such that for all $a \in G$, $e * a = a * e = a$.
4. **Inverse:** For each $a \in G$, there exists $a^{-1} \in G$ such that $a * a^{-1} = a^{-1} * a = e$.

If additionally $a * b = b * a$ for all $a, b \in G$, the group is called **abelian** (commutative).

Exam Alert

Memorise the four axioms with the mnemonic **CAII**: Closure, Associativity, Identity, Inverse. In an exam, always verify all four when checking if something is a group.

Example: Checking the Group Axioms

Is $(\mathbb{Z}, +)$ a group?

1. **Closure:** Sum of two integers is an integer. ✓
2. **Associativity:** $(a + b) + c = a + (b + c)$. ✓
3. **Identity:** $e = 0$, since $a + 0 = 0 + a = a$. ✓
4. **Inverse:** $a^{-1} = -a$, since $a + (-a) = 0 = e$. ✓

Also abelian since $a + b = b + a$. So $(\mathbb{Z}, +)$ is an **abelian group**. □

Example: Non-Example

Is $(\mathbb{N}, +)$ a group? No — natural numbers have no inverses (there is no natural number n such that $5 + n = 0$). Axiom 4 fails.

1.2 Addition Modulo Groups

Definition: Integers Modulo n

For a positive integer n , define $\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$ with the operation **addition modulo n** : $a +_n b = (a + b) \bmod n$.

Theorem: $\mathbb{Z}_n$ under Addition is an Abelian Group

For any $n \geq 1$, $(\mathbb{Z}_n, +_n)$ satisfies all four group axioms:

- **Closure**: $(a + b) \bmod n \in \{0, \dots, n-1\}$. ✓
- **Associativity**: Inherited from integer addition. ✓
- **Identity**: $e = 0$. ✓
- **Inverse**: Inverse of a is $n - a$ (for $a \neq 0$) and 0 (for $a = 0$). ✓
- **Abelian**: $a +_n b = b +_n a$. ✓

Example: $\mathbb{Z}_5$ Addition Table

$+_5$	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

Each row and column is a permutation of $\{0, 1, 2, 3, 4\}$ — the Latin square property.
Inverses: $0^{-1} = 0$, $1^{-1} = 4$, $2^{-1} = 3$, $3^{-1} = 2$, $4^{-1} = 1$.

1.3 Multiplication Modulo Groups

Definition: Multiplicative Group Modulo n

Define $\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n \mid \gcd(a, n) = 1\}$ with operation multiplication modulo n . This is the set of all elements of $\mathbb{Z}_n$ that are *coprime* to n .

Theorem: $\mathbb{Z}_n^*$ under Multiplication is a Group

$(\mathbb{Z}_n^*, \cdot_n)$ is a group under multiplication modulo n . The identity is 1. The inverse of a is the number b such that $a \cdot b \equiv 1 \pmod{n}$ (exists iff $\gcd(a, n) = 1$, guaranteed for elements of $\mathbb{Z}_n^*$).

Example: $\mathbb{Z}_8^*$

$\mathbb{Z}_8^* = \{1, 3, 5, 7\}$ (elements coprime to 8).

Multiplication table mod 8:

$\cdot_8$	1	3	5	7
1	1	3	5	7
3	3	1	7	5
5	5	7	1	3
7	7	5	3	1

Every element is its own inverse! This group is isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_2$.

Note: Euler's Totient Function

$\phi(n) = |\mathbb{Z}_n^*|$ is the **Euler totient function** — the count of integers in $\{1, \dots, n\}$ coprime to n .

- If p is prime: $\phi(p) = p - 1$.
- $\phi(p^k) = p^k - p^{k-1}$.
- $\phi(mn) = \phi(m)\phi(n)$ when $\gcd(m, n) = 1$.

1.4 Other Important Groups

Group	Set	Op	Identity	Abelian?
$(\mathbb{Z}, +)$	Integers	$+$	0	Yes
$(\mathbb{Q} \setminus \{0\}, \cdot)$	Non-zero rationals	$\cdot$	1	Yes
$(\mathbb{R} \setminus \{0\}, \cdot)$	Non-zero reals	$\cdot$	1	Yes
$(\mathbb{Z}_n, +_n)$	$\{0, \dots, n-1\}$	$+_n$	0	Yes
$(\mathbb{Z}_n^*, \cdot_n)$	Units mod n	$\cdot_n$	1	Yes
S_n (Symmetric group)	Permutations of n elements	Composition	Identity perm	No (for $n \geq 3$)
$GL_n(\mathbb{R})$	Invertible $n \times n$ matrices	Matrix mult	I_n	No (for $n \geq 2$)

1.5 Order of an Element and Order of a Group**Definition: Order of a Group**

The **order of a group** G , written $|G|$, is the number of elements in G .

Definition: Order of an Element

The **order of an element** $a \in G$, written $\text{ord}(a)$, is the *smallest positive integer* k such that $a^k = e$ (the identity).

If no such k exists, the order is infinite.

Note: Notation

a^k means $a * a * \cdots * a$ (k times). In additive notation, $a^k = ka$.

Example: Computing Orders in $\mathbb{Z}_6$

In $(\mathbb{Z}_6, +_6)$, the identity is 0. Compute $\text{ord}(2)$:

$$2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 6 \equiv 0 \pmod{6}.$$

So $\text{ord}(2) = 3$.

Similarly: $\text{ord}(1) = 6$, $\text{ord}(2) = 3$, $\text{ord}(3) = 2$, $\text{ord}(4) = 3$, $\text{ord}(5) = 6$, $\text{ord}(0) = 1$.

Theorem: Order Divides Group Order

For any element a in a finite group G : $\text{ord}(a)$ divides $|G|$. This follows from Lagrange's Theorem (see Section 4.7).

1.6 Cyclic Groups**Definition: Generator and Cyclic Group**

An element $a \in G$ is a **generator** of G if every element of G can be written as a^k for some integer k . A group G is **cyclic** if it has at least one generator. We write $G = \langle a \rangle$.

Theorem: Basic Properties of Cyclic Groups

1. Every cyclic group is **abelian**.
2. $\mathbb{Z}_n = \langle 1 \rangle$ — the group $\mathbb{Z}_n$ is cyclic, generated by 1.
3. The element $a \in \mathbb{Z}_n$ is a generator iff $\gcd(a, n) = 1$.
4. The number of generators of $\mathbb{Z}_n$ is $\phi(n)$.
5. Every subgroup of a cyclic group is itself cyclic.

Example: Finding Generators of $\mathbb{Z}_{10}$

$\phi(10) = \phi(2)\phi(5) = 1 \cdot 4 = 4$, so there are 4 generators.
Generators are elements a with $\gcd(a, 10) = 1$: $\{1, 3, 7, 9\}$.

Theorem: Cyclic Groups are Classified

- Every infinite cyclic group is isomorphic to $(\mathbb{Z}, +)$.
- Every finite cyclic group of order n is isomorphic to $(\mathbb{Z}_n, +_n)$.

1.7 Lagrange's Theorem and Corollaries

Definition: Subgroup

A subset $H \subseteq G$ is a **subgroup** of G (written $H \leq G$) if H is itself a group under the same operation. To verify, use the **subgroup test**:

- H is non-empty.
- For all $a, b \in H$: $a * b^{-1} \in H$ (closed under operation and inverses).

Definition: Coset

Let $H \leq G$ and $a \in G$. The **left coset** of H with representative a is:

$$aH = \{a * h \mid h \in H\}$$

Similarly, the right coset is $Ha = \{h * a \mid h \in H\}$.

Theorem: Lagrange's Theorem

Let G be a finite group and $H \leq G$ a subgroup. Then:

$$|H| \text{ divides } |G|$$

More precisely, $|G| = |H| \cdot [G : H]$ where $[G : H]$ is the number of distinct left cosets of H in G (called the **index** of H in G).

Example: Lagrange's Theorem in Practice

$G = \mathbb{Z}_6$, $|G| = 6$. Possible subgroup sizes are the divisors of 6: $\{1, 2, 3, 6\}$.
Indeed: $H_1 = \{0\}$ (size 1), $H_2 = \{0, 3\}$ (size 2), $H_3 = \{0, 2, 4\}$ (size 3), $H_4 = \mathbb{Z}_6$ (size 6).
There is **no subgroup of size 4 or 5**, since 4 and 5 do not divide 6.

Theorem: Corollaries of Lagrange's Theorem

1. **Order of element divides order of group:** $\text{ord}(a)$ divides $|G|$ for all $a \in G$.
2. **Groups of prime order are cyclic:** If $|G| = p$ (prime), then G is cyclic and every non-identity element is a generator.
3. **Fermat's Little Theorem:** If p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.
4. **Euler's Theorem:** If $\gcd(a, n) = 1$, then $a^{\phi(n)} \equiv 1 \pmod{n}$.

Example: Using Fermat's Little Theorem

Compute $2^{100} \bmod 7$.

Since 7 is prime: $2^6 \equiv 1 \pmod{7}$.

$100 = 6 \cdot 16 + 4$, so $2^{100} = (2^6)^{16} \cdot 2^4 \equiv 1^{16} \cdot 16 \equiv 16 \bmod 7 \equiv 2 \pmod{7}$.

1.8 Axioms of Ring Theory**Definition: Ring**

A **ring** is a set R with two binary operations $+$ (addition) and $\cdot$ (multiplication) satisfying:

1. $(R, +)$ is an **abelian group** (with identity 0).
2. **Associativity of multiplication:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
3. **Distributivity:** $a \cdot (b + c) = a \cdot b + a \cdot c$ and $(a + b) \cdot c = a \cdot c + b \cdot c$.

If also $a \cdot b = b \cdot a$ for all a, b , it is a **commutative ring**.

If there exists a multiplicative identity 1 with $1 \cdot a = a \cdot 1 = a$, it is a **ring with unity**.

Example: Examples of Rings

- $(\mathbb{Z}, +, \cdot)$ — integers. Commutative ring with unity.
- $(\mathbb{Z}_n, +_n, \cdot_n)$ — integers mod n . Commutative ring with unity.
- $M_{n \times n}(\mathbb{R})$ — $n \times n$ real matrices. Ring with unity, but *not* commutative.
- $\{0\}$ — trivial ring (zero ring).

Definition: Integral Domain and Zero Divisors

A **zero divisor** in a ring is a non-zero element a such that $a \cdot b = 0$ for some non-zero b .

A commutative ring with unity and **no zero divisors** is called an **integral domain**.

Example: Zero Divisors in $\mathbb{Z}_6$

$2 \cdot 3 = 6 \equiv 0 \pmod{6}$. So 2 and 3 are zero divisors in $\mathbb{Z}_6$.

$\mathbb{Z}_6$ is **not** an integral domain. However, $\mathbb{Z}_p$ for prime p **is** an integral domain (and even a field).

1.9 Polynomial Rings

Definition: Polynomial Ring $R[x]$

Let R be a ring. The **polynomial ring** $R[x]$ is the set of all polynomials with coefficients in R :

$$R[x] = \{a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \mid a_i \in R, n \geq 0\}$$

with the usual polynomial addition and multiplication.

Theorem: Properties of Polynomial Rings

- If R is a commutative ring, so is $R[x]$.
- If R is an integral domain, so is $R[x]$.
- Degree: $\deg(f \cdot g) = \deg(f) + \deg(g)$ (over an integral domain).
- **Division Algorithm:** For $f, g \in F[x]$ with F a field and $g \neq 0$, there exist unique $q, r \in F[x]$ with $f = gq + r$ and $\deg(r) < \deg(g)$.

Definition: Irreducible Polynomial

A polynomial $p(x) \in F[x]$ of degree ≥ 1 is **irreducible** over F if it cannot be factored as a product of two polynomials of lower degree with coefficients in F . (Analogous to prime numbers.)

Example: Irreducible over $\mathbb{Z}_2$

$x^2 + x + 1$ over $\mathbb{Z}_2$: test $x = 0$: $0 + 0 + 1 = 1 \neq 0$; $x = 1$: $1 + 1 + 1 = 3 \equiv 1 \neq 0$. No roots in $\mathbb{Z}_2$, so it is **irreducible** over $\mathbb{Z}_2$.

1.10 Axioms of Fields

Definition: Field

A **field** is a commutative ring with unity $(F, +, \cdot)$ in which every non-zero element has a multiplicative inverse:

$$\forall a \in F, a \neq 0 \Rightarrow \exists a^{-1} \in F : a \cdot a^{-1} = 1$$

Equivalently: $(F, +)$ is an abelian group, $(F \setminus \{0\}, \cdot)$ is an abelian group, and multiplication distributes over addition.

Structure	Extra axiom over ring	Examples
Ring	—	$\mathbb{Z}, \mathbb{Z}_n, M_n(\mathbb{R})$
Commutative Ring	$ab = ba$	$\mathbb{Z}, \mathbb{Z}_n, \mathbb{Q}[x]$
Integral Domain	No zero divisors	$\mathbb{Z}, \mathbb{Z}_p$ (p prime)
Field	Every non-zero has mult. inverse	$\mathbb{Q}, \mathbb{R}, \mathbb{C}, \mathbb{Z}_p$

Theorem: Finite Fields (Galois Fields)

A finite field of order q exists if and only if $q = p^n$ for some prime p and $n \geq 1$. It is written $\mathbb{F}_q$ or $GF(q)$.

Examples: $GF(2) = \mathbb{Z}_2$, $GF(4) = \mathbb{Z}_2[x]/(x^2 + x + 1)$, $GF(8) = \mathbb{Z}_2[x]/(x^3 + x + 1)$.

1.11 Primitive Elements**Definition: Primitive Element (Primitive Root)**

Let $G = (F^*, \cdot)$ be the multiplicative group of a finite field $F^* = F \setminus \{0\}$. A **primitive element** (or **primitive root**) $g \in F^*$ is a generator of this cyclic group:

$$F^* = \langle g \rangle = \{g^0, g^1, g^2, \dots, g^{q-2}\}$$

Theorem: Multiplicative Group of a Finite Field is Cyclic

For any finite field $GF(q)$, the multiplicative group $(GF(q)^*, \cdot)$ is cyclic. Therefore, primitive elements always exist.

Example: Primitive Root of $\mathbb{Z}_7^*$

$\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$, $|\mathbb{Z}_7^*| = 6$.

Check $g = 3$: $3^1 = 3$, $3^2 = 2$, $3^3 = 6$, $3^4 = 4$, $3^5 = 5$, $3^6 = 1 \pmod{7}$.

All 6 elements of $\mathbb{Z}_7^*$ appear, so 3 is a primitive root (generator) of $\mathbb{Z}_7^*$.

1.12 Applications in Cryptography**1.12.1 The Discrete Logarithm Problem****Definition: Discrete Logarithm Problem (DLP)**

Given a cyclic group G with generator g , a prime p , and an element $y \in G$, find the integer x such that:

$$g^x \equiv y \pmod{p}$$

x is called the **discrete logarithm** of y to base g modulo p .

Note: Why is DLP Hard?

Forward direction (easy): Given g, x, p , compute $g^x \pmod{p}$ quickly using *fast exponentiation* (square-and-multiply) in $O(\log x)$ multiplications.

Backward direction (hard): Given g, y, p , find x . No efficient classical algorithm is known for large p . This computational hardness is the foundation of many cryptographic protocols.

1.12.2 Diffie-Hellman Key Exchange

Definition: Diffie-Hellman Key Exchange Protocol

Allows two parties (Alice and Bob) to establish a **shared secret** over a public channel without ever transmitting the secret itself.

Setup (public): A large prime p and a primitive root g modulo p are published.

Protocol steps:

1. **Alice** picks a secret integer a , computes $A = g^a \bmod p$, and sends A to Bob.
2. **Bob** picks a secret integer b , computes $B = g^b \bmod p$, and sends B to Alice.
3. **Alice** computes the shared key: $K = B^a \bmod p = g^{ab} \bmod p$.
4. **Bob** computes the shared key: $K = A^b \bmod p = g^{ab} \bmod p$.

Both arrive at the same $K = g^{ab} \bmod p$.

Example: Diffie-Hellman (Small Numbers)

Public: $p = 23$, $g = 5$.

Alice: $a = 6$, sends $A = 5^6 \bmod 23 = 15625 \bmod 23 = 8$.

Bob: $b = 15$, sends $B = 5^{15} \bmod 23 = 19$.

Alice: $K = 19^6 \bmod 23 = 2$.

Bob: $K = 8^{15} \bmod 23 = 2$. ✓ Shared secret is $K = 2$.

Key Point

An eavesdropper sees p, g, A, B but must solve the DLP to find a or b . For p having 2048 bits, this is computationally infeasible with current technology.

2 Advanced Counting

2.1 Recurrence Relations of Recursion Algorithms

2.1.1 What is a Recurrence Relation?

Definition: Recurrence Relation

A **recurrence relation** defines each term of a sequence in terms of earlier terms. For algorithm analysis, $T(n)$ is the running time on input of size n , expressed as a function of T on smaller inputs.

Example: Factorial

factorial(n): if $n = 0$ return 1, else return $n \cdot \text{factorial}(n - 1)$.
 Recurrence: $T(n) = T(n - 1) + c$ (one recursive call + $O(1)$ work), with $T(0) = c_0$.
 Solution: $T(n) = nc + c_0 = O(n)$.

Example: Tower of Hanoi

Moving n disks: move top $n - 1$ to spare peg ($T(n - 1)$ moves), move bottom disk (1 move), move $n - 1$ back ($T(n - 1)$ moves).
 Recurrence: $T(n) = 2T(n - 1) + 1$, $T(1) = 1$.
 Solution by iteration:

$$\begin{aligned} T(n) &= 2T(n - 1) + 1 \\ &= 2(2T(n - 2) + 1) + 1 = 4T(n - 2) + 3 \\ &= 8T(n - 3) + 7 = \dots = 2^{n-1}T(1) + 2^{n-1} - 1 = 2^n - 1. \end{aligned}$$

2.1.2 Solving Linear Recurrences

Definition: Linear Homogeneous Recurrence with Constant Coefficients

Form: $T(n) = c_1T(n - 1) + c_2T(n - 2) + \dots + c_kT(n - k)$.
 To solve, find the **characteristic equation**:

$$r^k = c_1r^{k-1} + c_2r^{k-2} + \dots + c_k$$

Theorem: Solution from Characteristic Roots

- If the characteristic equation has k **distinct** roots $r_1, r_2, \dots, r_k$:

$$T(n) = A_1r_1^n + A_2r_2^n + \dots + A_kr_k^n$$

- If root r_i has **multiplicity** m : it contributes terms $(A_0 + A_1n + \dots + A_{m-1}n^{m-1})r_i^n$.
- Constants A_i are determined by **initial conditions**.

Example: Fibonacci Numbers

$$F(n) = F(n-1) + F(n-2), \quad F(0) = 0, \quad F(1) = 1.$$

Characteristic equation: $r^2 = r + 1 \Rightarrow r^2 - r - 1 = 0$.

Roots: $r = \frac{1 \pm \sqrt{5}}{2}$. Let $\phi = \frac{1 + \sqrt{5}}{2}$ (golden ratio), $\psi = \frac{1 - \sqrt{5}}{2}$.

General solution: $F(n) = A\phi^n + B\psi^n$.

From $F(0) = 0$: $A + B = 0 \Rightarrow B = -A$.

From $F(1) = 1$: $A\phi - A\psi = 1 \Rightarrow A(\phi - \psi) = A\sqrt{5} = 1 \Rightarrow A = \frac{1}{\sqrt{5}}$.

Binet's Formula: $F(n) = \frac{\phi^n - \psi^n}{\sqrt{5}}$.

2.2 Recurrence Relations of Divide and Conquer Algorithms**Definition: Divide and Conquer Recurrence**

When an algorithm divides a problem of size n into a subproblems of size n/b , and does $f(n)$ work outside the recursive calls:

$$T(n) = aT(n/b) + f(n), \quad T(1) = c$$

Example: Merge Sort

Divides array in half ($a = 2$, $b = 2$), merges in $O(n)$:

$$T(n) = 2T(n/2) + O(n)$$

Solution: $T(n) = O(n \log n)$.

Example: Binary Search

Checks midpoint ($O(1)$), recurses on one half ($a = 1$, $b = 2$):

$$T(n) = T(n/2) + O(1)$$

Solution: $T(n) = O(\log n)$.

2.3 Master's Theorem and Growth of Functions**Theorem: Master Theorem**

For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$:

1. **Case 1:** If $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$:

$$T(n) = \Theta(n^{\log_b a})$$

(Recursive work dominates.)

2. **Case 2:** If $f(n) = \Theta(n^{\log_b a})$:

$$T(n) = \Theta(n^{\log_b a} \log n)$$

(Both parts are equal.)

3. **Case 3:** If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and $af(n/b) \leq cf(n)$ for some $c < 1$:

$$T(n) = \Theta(f(n))$$

(Non-recursive work dominates.)

Example: Applying the Master Theorem

Recurrence	a	b	$\log_b a$	Case & Result
$T(n) = 2T(n/2) + n$	2	2	1	Case 2: $\Theta(n \log n)$
$T(n) = 2T(n/2) + 1$	2	2	1	Case 1: $\Theta(n)$
$T(n) = 4T(n/2) + n$	4	2	2	Case 1: $\Theta(n^2)$
$T(n) = T(n/2) + n$	1	2	0	Case 3: $\Theta(n)$
$T(n) = 3T(n/3) + n \log n$	3	3	1	Case 2*: $\Theta(n \log^2 n)$

*Extended Case 2 with log factors.

Exam Alert

The Master Theorem does NOT apply when: (a) $a < 1$ or $b \leq 1$, (b) $f(n)$ is not polynomially larger/smaller than $n^{\log_b a}$ (e.g. $f(n) = n^{\log_b a} / \log n$ falls between cases 1 and 2). In such cases, use substitution or the Akra-Bazzi method.

2.4 Generating Functions

Definition: Ordinary Generating Function

The **ordinary generating function (OGF)** of a sequence $a_0, a_1, a_2, \dots$ is the formal power series:

$$G(x) = \sum_{n=0}^{\infty} a_n x^n = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots$$

We treat x as a formal variable; convergence is not required.

Note: Key OGF Pairs to Memorise

$$\frac{1}{1-x} = \sum_{n=0}^{\infty} x^n = 1 + x + x^2 + \dots$$

$$\frac{1}{1-ax} = \sum_{n=0}^{\infty} a^n x^n$$

$$\frac{1}{(1-x)^2} = \sum_{n=0}^{\infty} (n+1)x^n$$

$$\frac{1}{(1-x)^k} = \sum_{n=0}^{\infty} \binom{n+k-1}{k-1} x^n$$

$$(1+x)^n = \sum_{k=0}^n \binom{n}{k} x^k$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

2.5 Applications of Generating Functions in Counting**Theorem: Coefficient Extraction**

The number of ways to count objects with given constraints is often the coefficient $[x^n] G(x)$ of a generating function.

Example: Distributing Identical Objects

How many ways to distribute n identical balls into 3 distinct boxes?

Each box can receive $0, 1, 2, \dots$ balls. The GF for one box is $\frac{1}{1-x}$.

For 3 boxes: $G(x) = \left(\frac{1}{1-x}\right)^3 = \frac{1}{(1-x)^3} = \sum_{n=0}^{\infty} \binom{n+2}{2} x^n$.

Answer: $[x^n] G(x) = \binom{n+2}{2}$.

Example: Restricted Distributions

How many ways to distribute n identical balls into 3 boxes with each box receiving *at least 1* ball?

Each box: $x + x^2 + x^3 + \dots = \frac{x}{1-x}$.

$G(x) = \left(\frac{x}{1-x}\right)^3 = \frac{x^3}{(1-x)^3}$.

$[x^n] G(x) = [x^{n-3}] \frac{1}{(1-x)^3} = \binom{n-3+2}{2} = \binom{n-1}{2}$.

2.6 Applications of Generating Functions in Solving Recurrence Relations

Example: Solving a Recurrence with GFs

Solve $a_n = 3a_{n-1} + 2$, $a_0 = 1$.

Step 1: Multiply both sides by x^n and sum over $n \geq 1$:

$$\sum_{n \geq 1} a_n x^n = 3x \sum_{n \geq 1} a_{n-1} x^{n-1} + 2 \sum_{n \geq 1} x^n$$

$$G(x) - a_0 = 3x G(x) + \frac{2x}{1-x}$$

Step 2: Solve for $G(x)$:

$$G(x)(1-3x) = 1 + \frac{2x}{1-x} = \frac{1-x+2x}{1-x} = \frac{1+x}{1-x}$$

$$G(x) = \frac{1+x}{(1-x)(1-3x)}$$

Step 3: Partial fractions: $\frac{1+x}{(1-x)(1-3x)} = \frac{A}{1-x} + \frac{B}{1-3x}$.

Solving: $A = -1$, $B = 2$. So:

$$G(x) = \frac{-1}{1-x} + \frac{2}{1-3x} = -\sum x^n + 2 \sum 3^n x^n$$

$$a_n = 2 \cdot 3^n - 1$$

Check: $a_0 = 2 - 1 = 1$. ✓ $a_1 = 3(1) + 2 = 5 = 2(3) - 1 = 5$. ✓

Key Point

General strategy for GF solution of recurrences:

1. Define $G(x) = \sum a_n x^n$.
2. Multiply the recurrence by x^n and sum over relevant n .
3. Use initial conditions to get a closed form for $G(x)$.
4. Expand $G(x)$ via partial fractions or known series to read off a_n .

3 Graph Theory

3.1 Graph Terminologies

Definition: Graph

A **graph** $G = (V, E)$ consists of:

- A non-empty set V of **vertices** (also called nodes).
- A set E of **edges**, where each edge connects two vertices.

The number of vertices $|V|$ is the **order** of the graph; $|E|$ is its **size**.

Term	Meaning
Adjacent / Neighbours	Two vertices connected by an edge
Incident	A vertex and an edge are incident if the vertex is an endpoint
Degree $\deg(v)$	Number of edges incident to v
Loop	An edge from a vertex to itself
Multiple edges	More than one edge between the same pair of vertices
Simple graph	No loops, no multiple edges
Walk	A sequence of vertices and edges (may repeat)
Trail	A walk with no repeated edge
Path	A walk with no repeated vertex
Cycle	A closed path (starts and ends at same vertex, no repeats)
Connected	A path exists between every pair of vertices
Component	Maximal connected subgraph

Theorem: Handshaking Lemma

For any graph $G = (V, E)$:

$$\sum_{v \in V} \deg(v) = 2|E|$$

Proof: Each edge contributes exactly 2 to the total degree sum (one for each endpoint).

Corollary 3.1. *The number of vertices with **odd degree** is always even.*

Example: Handshaking

A graph has 6 vertices with degrees 1, 2, 3, 3, 4, 5. Sum = 18 = 2×9 , so it has 9 edges. Odd-degree vertices: 1, 3, 3, 5 — four of them (even count). ✓

3.2 Types of Graphs

Type	Description
Simple Graph	No loops, no multiple edges
Multigraph	Allows multiple edges (but no loops)
Pseudograph	Allows both loops and multiple edges
Directed Graph (Digraph)	Edges have direction (ordered pairs)
Weighted Graph	Each edge has an associated numerical weight
Undirected Graph	Edges have no direction
Complete Graph K_n	Every pair of vertices is connected
Bipartite Graph	Vertices split into two sets; edges only go between sets
Tree	Connected, acyclic graph
Forest	Acyclic graph (possibly disconnected)
Planar Graph	Can be drawn in a plane with no edge crossings

3.2.1 Subgraphs

Definition: Subgraph

$H = (V', E')$ is a **subgraph** of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$ with all endpoints of E' in V' .

A **spanning subgraph** has $V' = V$ (uses all vertices).

An **induced subgraph** $G[V']$ contains all edges of G between vertices in V' .

3.3 Some Common Graphs

3.3.1 Complete Graph K_n

Definition: K_n — Complete Graph

K_n has n vertices and an edge between **every** pair of vertices.

$$|V| = n, \quad |E| = \binom{n}{2} = \frac{n(n-1)}{2}$$

Every vertex has degree $n - 1$. (K_n is $(n - 1)$ -regular.)

Graph	Vertices	Edges
K_1	1	0
K_2	2	1
K_3	3	3
K_4	4	6
K_5	5	10
K_n	n	$n(n-1)/2$

3.3.2 Cycle Graph C_n

Definition: C_n — Cycle Graph

C_n has n vertices arranged in a single cycle.

$$|V| = n, \quad |E| = n \quad (n \geq 3)$$

Every vertex has degree 2. (C_n is 2-regular.)

3.3.3 Wheel Graph W_n

Definition: W_n — Wheel Graph

W_n is obtained by adding a single **hub vertex** connected to all vertices of C_n .

$$|V| = n + 1, \quad |E| = 2n$$

The hub has degree n ; the rim vertices have degree 3.

3.3.4 Hypercube Graph Q_n

Definition: Q_n — n -dimensional Hypercube

Q_n has one vertex for each n -bit binary string. Two vertices are adjacent iff their binary labels differ in **exactly one bit**.

$$|V| = 2^n, \quad |E| = n \cdot 2^{n-1}$$

Every vertex has degree n (Q_n is n -regular).

Example: Q_3

Vertices: 000, 001, 010, 011, 100, 101, 110, 111 (8 vertices).

000 is adjacent to 001, 010, 100 (differ in one bit). Each vertex has degree 3.

$$|E| = 3 \times 2^2 = 12.$$

3.3.5 Complete Bipartite Graph $K_{m,n}$

Definition: $K_{m,n}$ — Complete Bipartite Graph

$K_{m,n}$ has vertex sets V_1 (size m) and V_2 (size n) with an edge between *every* vertex in V_1 and *every* vertex in V_2 (no edges within the same set).

$$|V| = m + n, \quad |E| = mn$$

Vertices in V_1 have degree n ; vertices in V_2 have degree m .

Graph	Vertices	Edges
$K_{1,n}$ (Star)	$n + 1$	n
$K_{2,3}$	5	6
$K_{3,3}$	6	9
$K_{m,n}$	$m + n$	mn

3.4 Bipartite Graphs

Definition: Bipartite Graph

A graph $G = (V, E)$ is **bipartite** if the vertex set V can be partitioned into two disjoint sets V_1 and V_2 such that every edge has one endpoint in V_1 and the other in V_2 .

Theorem: Characterisation of Bipartite Graphs

A graph is bipartite if and only if it contains **no odd-length cycle**.

Intuition: In a bipartite graph you alternate between V_1 and V_2 as you traverse a cycle. To return to the start, you need an even number of steps.

Example: Testing Bipartiteness (2-Colouring)

Algorithm: BFS/DFS with 2 colours. Assign one colour to the start vertex; assign the opposite colour to each neighbour. If a conflict arises (adjacent vertices get the same colour), the graph is **not** bipartite; otherwise it **is** bipartite.

Example: Is C_6 Bipartite?

C_6 (even cycle): $V_1 = \{v_1, v_3, v_5\}$, $V_2 = \{v_2, v_4, v_6\}$. Every edge goes $V_1 \leftrightarrow V_2$. Yes, bipartite. ✓

C_5 (odd cycle): Cannot 2-colour without conflict. **Not** bipartite.

Note: Common Bipartite Graphs

$K_{m,n}$, all trees, C_{2k} (even cycles), Q_n (hypercubes).

All of K_3, C_5, K_4 (non-bipartite): contain odd cycles.

3.5 Euler Paths and Hamiltonian Paths

3.5.1 Euler Paths and Circuits

Definition: Euler Path and Circuit

- An **Euler path** (or Eulerian trail) is a **trail** (no repeated edges) that visits **every edge** exactly once.
- An **Euler circuit** is an Euler path that starts and ends at the **same vertex**.

Theorem: Euler Circuit Theorem — Euler 1736

A connected graph G has an **Euler circuit** if and only if **every vertex has even degree**.

Theorem: Euler Path Theorem

A connected graph G has an **Euler path** (but not a circuit) if and only if it has **exactly two vertices of odd degree**. The path starts at one odd-degree vertex and ends at the other.

Example: The Königsberg Bridge Problem

The original problem (Euler, 1736): can you cross all 7 bridges of Königsberg exactly once?

Model as a graph: 4 landmasses (vertices) and 7 bridges (edges).

Degrees: 3, 3, 3, 5 — all four vertices have **odd degree**.

For an Euler path we need exactly 2 odd-degree vertices; for an Euler circuit, 0.

Having 4 odd-degree vertices means **no Euler path exists**. It is impossible.

Example: Deciding Euler Path/Circuit

Graph	Odd-degree vertices	Euler Circuit?	Euler Path?
K_4	0 (all degree 3... wait, degree is 3 odd!)	No	No (4 odd)
K_4 corrected	all have degree 3: 4 odd-degree vertices	No	No
K_5	0 (all degree 4)	Yes	—
C_n	0 (all degree 2)	Yes	—
$K_{2,3}$	4 odd-degree vertices	No	No
Path P_n	2 (the two endpoints, degree 1)	No	Yes

3.5.2 Fleury's Algorithm (Finding an Euler Circuit)

1. Start at any vertex (any odd-degree vertex if finding an Euler path).
2. At each step, choose an edge that is **not a bridge** (an edge whose removal disconnects the graph), unless there is no other choice.
3. Remove the traversed edge. Continue until all edges are used.

3.5.3 Hamiltonian Paths and Circuits**Definition: Hamiltonian Path and Circuit**

- A **Hamiltonian path** is a path that visits **every vertex** exactly once.
- A **Hamiltonian circuit** is a Hamiltonian path that starts and ends at the **same vertex**.

Exam Alert

Unlike Euler paths (which have a clean necessary and sufficient condition), there is **no known efficient characterisation** of Hamiltonian graphs. Deciding whether a Hamiltonian circuit exists is an **NP-complete** problem.

Theorem: Dirac's Theorem (Sufficient Condition)

If G is a simple graph with $n \geq 3$ vertices and every vertex has degree $\deg(v) \geq n/2$, then G has a Hamiltonian circuit.

Theorem: Ore's Theorem (Sufficient Condition)

If G is a simple graph with $n \geq 3$ vertices and for every pair of *non-adjacent* vertices u, v : $\deg(u) + \deg(v) \geq n$, then G has a Hamiltonian circuit.

Note: These are Sufficient Conditions Only — Not Necessary

A graph can have a Hamiltonian circuit even if neither Dirac's nor Ore's condition holds. These theorems only guarantee existence when the condition is met; they cannot be used to prove non-existence.

3.5.4 Euler vs Hamiltonian: Key Comparison

Property	Euler	Hamiltonian
Visits	Every edge once	Every vertex once
Key condition	Vertex degrees (even/odd)	No clean condition
Characterisation	Clean (Euler's theorem)	None (NP-complete to decide)
Algorithm	Efficient (Fleury's, Hierholzer's)	No efficient algorithm known
Circuit condition	All degrees even	No known simple condition
Path condition	Exactly 2 odd-degree vertices	No known simple condition

Example: Hamiltonian Circuit in K_n

K_n always has a Hamiltonian circuit for $n \geq 3$: just visit all vertices in any order and return to start. The number of distinct Hamiltonian circuits in K_n is $\frac{(n-1)!}{2}$.

Example: No Hamiltonian Circuit in $K_{1,3}$ (Star)

$K_{1,3}$ has a centre vertex connected to 3 leaves. Any path starting from a leaf must go through the centre, then to another leaf — but then it cannot visit the third leaf and return to start without revisiting the centre. So $K_{1,3}$ has **no Hamiltonian circuit**.

3 Quick Revision Summary (Topics 4–6)

Topic 4: Abstract Algebra

- Group axioms: **CAII** — Closure, Associativity, Identity, Inverse.
- Abelian group adds commutativity. $(\mathbb{Z}_n, +_n)$ is always an abelian group.
- $\mathbb{Z}_n^*$ = elements coprime to n ; $|\mathbb{Z}_n^*| = \phi(n)$ (Euler totient).
- Order of element a : smallest k with $a^k = e$. Divides $|G|$ (Lagrange).
- Cyclic group: generated by one element. $\mathbb{Z}_n$ is cyclic; generators are elements coprime to n .
- **Lagrange**: $|H|$ divides $|G|$ for any subgroup $H \leq G$.
- **Fermat's Little Theorem**: $a^{p-1} \equiv 1 \pmod{p}$ (p prime, $\gcd(a, p) = 1$).
- Ring: abelian group under $+$, associative $\cdot$, distributive. Field: ring where every non-zero element has a mult. inverse.
- Irreducible polynomial: like a prime in polynomial ring.
- Primitive root g : generates all of $\mathbb{Z}_p^*$. DLP: find x given $g^x \equiv y \pmod{p}$.
- Diffie-Hellman: Alice sends g^a , Bob sends g^b ; shared key = g^{ab} .

Topic 5: Advanced Counting

- Recurrence from recursion: one call of size $n-1$ gives $T(n) = T(n-1) + f(n)$.
- Characteristic equation for linear homogeneous recurrences: substitute r^n .
- Fibonacci: $F(n) = F(n-1) + F(n-2)$; closed form via Binet's formula.
- D&C recurrence: $T(n) = aT(n/b) + f(n)$. Solve with Master Theorem.
- Master Theorem: compare $f(n)$ with $n^{\log_b a}$. Three cases: Case 1 (recursive dominates), Case 2 (equal), Case 3 (non-recursive dominates).
- OGF: $G(x) = \sum a_n x^n$. Key series: $\frac{1}{1-x}$, $\frac{1}{(1-x)^k}$, $(1+x)^n$.
- GF for counting: multiply GFs of independent choices; read off coefficient $[x^n]$.
- GF for recurrences: write $G(x)$, use recurrence to form equation, solve, expand via partial fractions.

Topic 6: Graph Theory

- Graph $G = (V, E)$. Handshaking: $\sum \deg(v) = 2|E|$; odd-degree count is even.
- K_n : $n(n-1)/2$ edges, $(n-1)$ -regular. C_n : n edges, 2-regular. W_n : $n+1$ vertices, $2n$ edges. Q_n : 2^n vertices, $n \cdot 2^{n-1}$ edges, n -regular. $K_{m,n}$: mn edges.
- Bipartite $\Leftrightarrow$ no odd cycle $\Leftrightarrow$ 2-colourable. Test with BFS.
- **Euler circuit**: exists iff connected + all degrees even.
- **Euler path**: exists iff connected + exactly 2 odd-degree vertices.
- Königsberg: 4 odd-degree vertices $\Rightarrow$ neither Euler path nor circuit.
- **Hamiltonian**: visits every vertex once. No clean characterisation (NP-complete).
- Dirac: $\deg(v) \geq n/2$ for all $v \Rightarrow$ Hamiltonian circuit exists.
- Ore: $\deg(u) + \deg(v) \geq n$ for all non-adjacent $u, v \Rightarrow$ Hamiltonian circuit exists.